

Atelier Préparation de Données Images

Contexte

Une entreprise souhaite développer un système d'intelligence artificielle capable de reconnaître automatiquement le type de déchet présent sur une photographie afin d'améliorer le tri des déchets.

Le modèle devra classer chaque image dans l'une des catégories suivantes :

- cardboard : cartons ondulés, cartons plats, ...
- plastic : bouteilles, emballages plastiques...
- paper : feuilles, journaux...
- glass : bouteilles et objets en verre...
- metal : canettes, boîtes métalliques...
- trash : emballages bonbons, tasses jetables, ...

Le problème est que les images collectées proviennent de plusieurs sources. Elles ne sont donc pas homogènes : dimensions différentes ; formats différents ; images RGB et grayscale ; certaines images sont trop petites ; certaines images sont corrompues ; quelques images sont vides ; images dupliquées ; quelques images placées dans le mauvais dossier ; classes déséquilibrées.

L'objectif de l'atelier est donc de construire un jeu de données images propre et homogène, prêt à être utilisé pour entraîner un modèle de Machine Learning ou de Deep Learning.

Objectifs pédagogiques

À la fin de l'atelier, l'apprenant devra être capable de :

- 1) explorer un dataset d'images ;
- 2) détecter les images problématiques ;
- 3) détecter les différences de résolution ;
- 4) détecter les différences de nombre de canaux ;
- 5) identifier les images trop petites ;
- 6) détecter les doublons ;
- 7) identifier les classes déséquilibrées ;
- 8) redimensionner les images ;
- 9) normaliser les valeurs des pixels ;
- 10) uniformiser les canaux ;
- 11) appliquer de la data augmentation

Structure du projet

1) Structurer le projet comme suit :

```
atelier_prepa_donnees_images/  
├── notebooks/  
│   └── atelier_prepa_donnees_images.ipynb  
├── reports/  
│   └── audit_images.csv  
└── data/  
    ├── raw/  
    │   ├── cardboard/  
    │   ├── glass/  
    │   ├── metal/  
    │   ├── paper/  
    │   ├── plastic/  
    │   └── trash/  
    └── cleaned/  
        ├── cardboard/  
        ├── glass/  
        ├── metal/  
        ├── paper/  
        ├── plastic/  
        └── trash/
```

2) Stocker les images fournies dans le sous-dossier `raw/`

3) Créer le notebook `atelier_prepa_donnees_images.ipynb`

NB 1 : le dossier `cleaned` sera créé lors des modifications (dataset original doit être en lecture seule)

NB 2: le fichier `audit_images.csv` contiendra des résultats de l'audit du dataset comme le nombre : "total d'images", "Images corrompues", "Images quasi vides", "Images trop petites", "Doublons", "Images grayscale", "Images RGBA", "images mal classées"

Partie 1 – Exploration du dataset

Développer un programme Python capable de récupérer, pour chaque image, son nom, sa classe, son format, son mode, sa largeur, sa hauteur, l'écart-type de ses pixels, son nombre de canaux et sa taille.

NB : prendre en charge aussi les fichiers corrompus

Partie 2 – Détecter les images corrompues

Écrire et se servir d'une fonction qui détecte une image corrompue.

Partie 3 – Détecter les images vides

Écrire et se servir d'une fonction qui détecte les images vides : image entièrement noire, image entièrement blanche ou image dont les pixels présentent très peu de variation.

Partie 4 – Détecter les différences de résolution

- 1) Déterminer la résolution minimale, la résolution maximale, les résolutions les plus fréquentes et le nombre d'images par résolution.
- 2) On décide qu'une image doit avoir au minimum 64×64 pixels. Identifier toutes les images ne respectant pas cette contrainte.

Partie 5 – Détecter les différents canaux

Déterminer le nombre d'images selon leur nombre de canaux.

Partie 6 – Détecter les doublons

Écrire et se servir d'une fonction qui détecte les doublons

NB : Deux fichiers différents peuvent contenir exactement la même image.

Partie 7 : Détecter les images mal classées

Effectuer un contrôle visuel pour détecter toute image mal classée.

Partie 8 : Analyser le déséquilibre des classes

Déterminer le nombre d'images par classe

Partie 9 : Redimensionnement

Redimensionner toutes les images à 224×224 .

NB : Redimensionnement avec conservation des proportions et ajout éventuel de padding

Partie 10 : Uniformisation des canaux

Convertir toutes les images en RGB

Partie 11 : Mise à l'échelle des pixels

Normaliser les valeurs des pixels de toutes les images

Partie 12 : Découpage Train/Validation/Test

Découper le dataset nettoyé (cleaned) en trois sous-ensembles : train, validation et test

Partie 13 : Data Augmentation

- 1) A l'aide de Keras, appliquer à la classe minoritaire de la « data augmentation » en utilisant des transformations comme rotation légère, retournement horizontal, zoom, translation, variation de luminosité, variation de contraste, légère variation de couleur, ...
- 2) Pourquoi éviter de faire la « data augmentation » avant le découpage du dataset ?

Partie 14 : Bonus

Proposer et réaliser toute fonctionnalité pertinente et utile qu'on pourrait rajouter à l'atelier.

Livrable attendu

Chaque apprenant devra fournir le dossier `atelier_prepa_donnees_images`

Modalités de livraison

Chaque apprenant pousse

- 1) le livrable, **d'abord vide**, sur un dépôt public de son compte Github personnel ;
- 2) met à jour le livrable **après chaque tâche accomplie** (sans attendre de terminer toute une partie), avec un message de commit explicite.